

Structural Patterns - Part 2

Facade | Flyweight | Proxy

Sub-Chapter 02-04 | Simplest Examples | With Diagrams

Facade	Simplify a complex subsystem with one easy interface
Flyweight	Share common data to save memory
Proxy	Control access to an object

1 Facade Pattern

Simplify a complex subsystem with one easy interface

What is it?

You have a complex home theater system: TV, SoundBar, Lights, and a BluRay player. Instead of controlling each device separately (calling `tv.on()`, `sound.on()`, `lights.dim()` every time), you create a single **HomeTheater** facade that does everything for you with one call: **`watchMovie()`**.

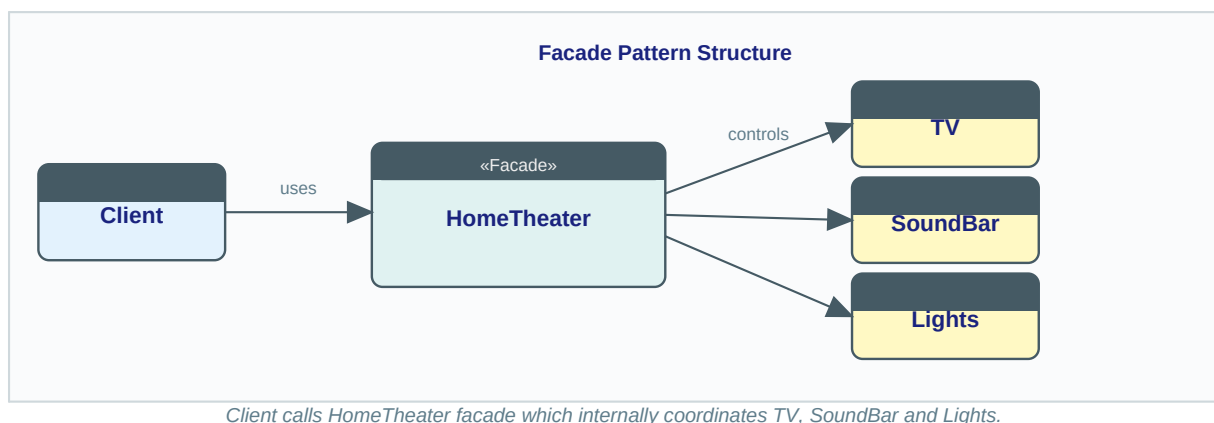
Simple Java Example

```
class TV { void on() { System.out.println("TV on"); } void off() { System.out.println("TV off"); } }
class SoundBar { void on() { System.out.println("Sound on"); } void off() { System.out.println("Sound off"); } }
class Lights { void dim() { System.out.println("Lights dim"); } void bright() { System.out.println("Lights bright"); } }

// Facade
class HomeTheater {
    TV tv = new TV(); SoundBar sound = new SoundBar(); Lights lights = new Lights();
    void watchMovie() {
        lights.dim(); tv.on(); sound.on();
        System.out.println("--- Enjoy the movie! ---");
    }
    void endMovie() {
        sound.off(); tv.off(); lights.bright();
        System.out.println("--- Movie ended ---");
    }
}

// Client only talks to facade
HomeTheater ht = new HomeTheater();
ht.watchMovie();
ht.endMovie();
```

Diagram



Key Rules

- The Facade wraps many complex classes behind a single simple interface.
- The client never needs to know about the subsystem classes (TV, SoundBar, Lights).
- Subsystem classes can still be used directly if needed - the facade does not lock them away.

- One facade can provide multiple simplified operations (watchMovie, endMovie, etc.).

Quick Q&A

Q1. What problem does Facade solve?

It hides a complex subsystem behind a single simple interface so the client code stays clean and easy to read.

Q2. How is Facade different from Adapter?

Adapter makes two incompatible interfaces work together. Facade simplifies a group of interfaces into one easier interface.

Q3. Can you still use subsystem classes directly when a Facade exists?

Yes. The Facade is just a convenience layer; the subsystem classes are still accessible directly.

Q4. When should you use Facade?

When you have many interacting classes and you want to give the client a simple, high-level API without exposing all the internals.

Q5. Does Facade add new behaviour or just delegate?

Mainly delegates - it orchestrates calls to subsystem objects but does not change what those objects do.

2 Flyweight Pattern

Share common data to save memory

What is it?

Imagine drawing **1 million trees** in a game. Every tree has a unique position (x, y) but most trees share the same colour and texture (e.g. all oak trees look the same). Instead of storing colour and texture in every single Tree object, we store that shared data **once** in a **TreeType** flyweight and let all oak Tree objects point to it.

Simple Java Example

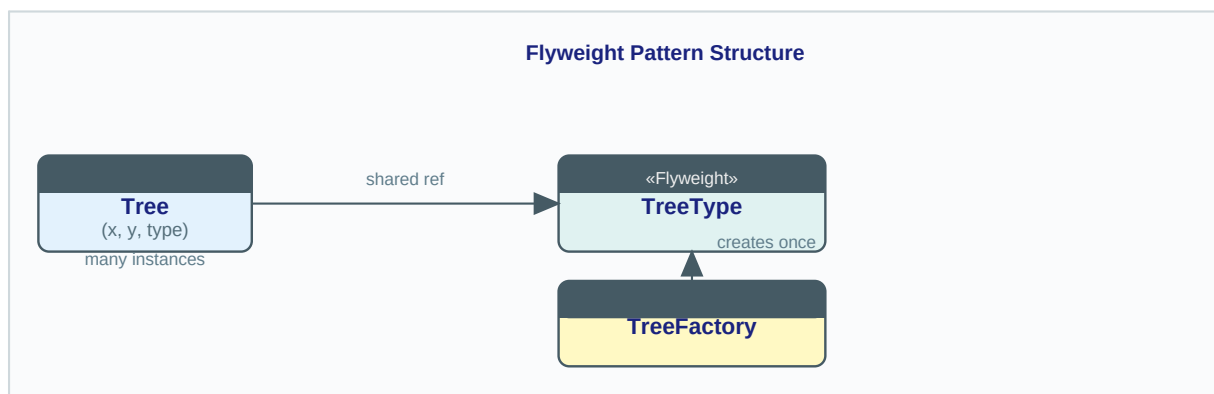
```
// Flyweight -- shared (intrinsic) data
class TreeType {
    String name, colour;
    TreeType(String name, String colour) { this.name=name; this.colour=colour; }
    void draw(int x, int y) {
        System.out.println(name + " [" + colour + "] at (" + x + ", " + y + ")");
    }
}

// Context -- unique (extrinsic) data per tree instance
class Tree {
    int x, y; TreeType type;
    Tree(int x, int y, TreeType type) { this.x=x; this.y=y; this.type=type; }
    void draw() { type.draw(x, y); }
}

// Factory -- ensures each TreeType is created only once
class TreeFactory {
    static Map<String,TreeType> cache = new HashMap<>();
    static TreeType get(String name, String colour) {
        return cache.computeIfAbsent(name, k -> new TreeType(name, colour));
    }
}

TreeType oak = TreeFactory.get("Oak", "Green");
Tree t1 = new Tree(10, 20, oak); Tree t2 = new Tree(30, 45, oak);
t1.draw(); t2.draw();
System.out.println(t1.type == t2.type); // true -- same shared object
```

Diagram



Many Tree objects share one TreeType flyweight. TreeFactory ensures the flyweight is created only once.

Key Rules

- Split state into intrinsic (shared, stored in flyweight) and extrinsic (unique, passed in or stored in context).
- A factory ensures each distinct flyweight is created only once and reused.
- Saves huge amounts of memory when you have thousands or millions of similar objects.
- The trade-off: slightly more complex code; worth it only when memory is a real concern.

Quick Q&A

Q1. What is intrinsic state?

Data that is the same across many objects and can be shared -- e.g. the colour and texture of a tree type.

Q2. What is extrinsic state?

Data that is unique to each object instance -- e.g. the x and y coordinates of each individual tree.

Q3. Why use a factory with Flyweight?

The factory acts as a cache: it returns an existing flyweight if one already exists for that key, preventing duplicates.

Q4. When should you use Flyweight?

When your application creates a very large number of similar objects and memory usage becomes a problem.

Q5. Does Flyweight change the behaviour of the objects?

No -- the objects behave exactly the same. Flyweight is purely an optimisation for memory, not a change in functionality.

3 Proxy Pattern

Control access to an object

What is it?

A **Proxy** sits in front of a real object and controls access to it. Think of a bank account: a proxy checks your PIN before allowing any withdrawal. The client talks to the proxy through the same interface as the real object, so it does not even know a proxy is in the middle.

Simple Java Example

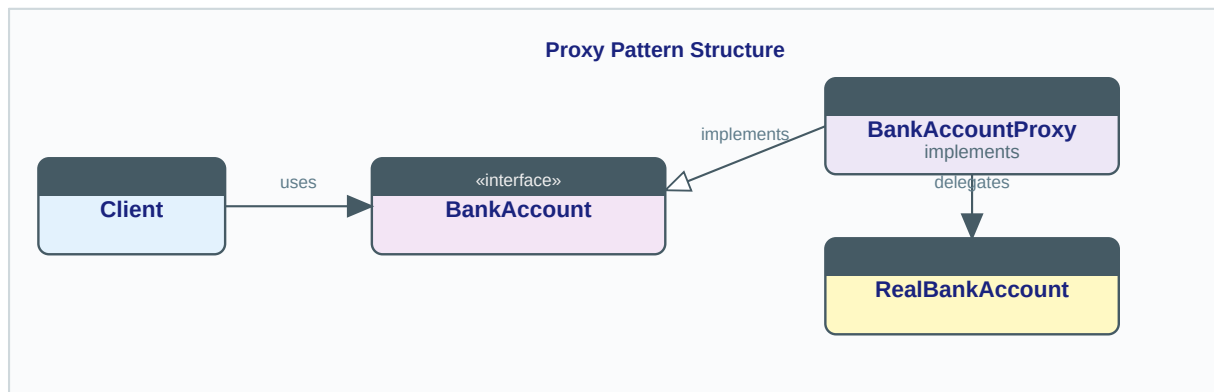
```
interface BankAccount { void withdraw(int amount); }

// Real object
class RealBankAccount implements BankAccount {
    private int balance = 1000;
    public void withdraw(int amount) {
        balance -= amount;
        System.out.println("Withdrew " + amount + ". Balance: " + balance);
    }
}

// Proxy -- adds PIN check before delegating
class BankAccountProxy implements BankAccount {
    private RealBankAccount real = new RealBankAccount();
    private int userPin;
    BankAccountProxy(int pin) { this.userPin = pin; }
    public void withdraw(int amount) {
        if (userPin == 1234) real.withdraw(amount);
        else System.out.println("Wrong PIN! Access denied.");
    }
}

// Usage
BankAccount acc1 = new BankAccountProxy(1234); // correct PIN
acc1.withdraw(200); // Withdrew 200. Balance: 800
BankAccount acc2 = new BankAccountProxy(9999); // wrong PIN
acc2.withdraw(100); // Wrong PIN! Access denied.
```

Diagram



Client uses BankAccount interface. Proxy intercepts calls and delegates to RealBankAccount only after PIN check.

Key Rules

- The proxy implements the same interface as the real object so the client code needs no changes.
- The client does not know (and does not care) whether it is talking to a proxy or the real thing.
- Common proxy uses: security check (Protection Proxy), lazy loading (Virtual Proxy), caching, logging.
- The proxy holds a reference to the real object and delegates after performing its extra logic.

Quick Q&A

Q1. What is the key design rule for a Proxy?

The proxy must implement the same interface as the real subject so the client can use either interchangeably.

Q2. How is Proxy different from Decorator?

Both wrap an object, but Proxy controls access to it while Decorator adds new behaviour/features to it.

Q3. Name three common types of Proxy.

Protection Proxy (security checks), Virtual Proxy (lazy initialisation), Caching Proxy (stores results to avoid repeated work).

Q4. Does the client know it is using a Proxy?

No -- the client only sees the interface. The proxy is transparent from the client's perspective.

Q5. When would you use a Virtual Proxy?

When creating the real object is expensive and you want to delay creation until it is actually needed.

Cheat Sheet -- Structural Patterns Part 2

Pattern	Intent	Key Trick	Real Java Example	Spot Violation
Facade	Provide a simple interface to a complex subsystem	One class wraps many; client calls only the facade	javax.faces.context.FacesContext wraps the JSF subsystem	Client still imports and calls subsystem classes directly
Flyweight	Share common state to reduce memory for many similar objects	Split intrinsic (shared) vs extrinsic (unique) state; use a factory cache	java.lang.Integer.valueOf() caches -128 to 127	Every object stores its own copy of shared data (no caching)
Proxy	Control access to an object (security, lazy load, cache, log)	Proxy implements same interface; delegates after extra logic	java.lang.reflect.Proxy; Spring AOP proxies	Client creates real object directly, bypassing all access control